

PART I

```
1832 ptcb = OSTCBLIST;
1833 INT16U MIN_ID=0xFFFF, MAX_Arrived= 0xFFFF;
1834 while (ptcb != (OSTCB*)0) {
1835     INT8U prio = ptcb->OSTCBPrio;
1836     INT8U y = prio >> 3;
1837     INT8U x = prio & 0x07;
1838
1839     INT8U maskY = 1 << y;
1840     INT8U maskX = 1 << x;
1841
1842     INT8U InRdyGrp = (OSRdyGrp & maskY) != 0;
1843     INT8U InRdyTbl = (OSRdyTbl[y] & maskX) != 0;
1844
1845     if (OSTime == 0) {
1846         if (InRdyGrp && InRdyTbl) {
1847             if (DEBUG)
1848                 printf("Task @%p (prio=%d) is READY %d %d %d\n", ptcb, ptcb->OSTCBID, ptcb->TaskPeriodic, ptcb->OSTCBID, ptcb->TaskPeriodic - ptcb->OSTCBID);
1849             if (ptcb->OSTCBID < MAX_Arrived) {
1850                 MAX_Arrived = ptcb->OSTCBID;
1851                 MIN_ID = ptcb->OSTCBID;
1852                 OSPrioHighRdy = prio;
1853             }
1854             else if (ptcb->OSTCBID == MAX_Arrived && ptcb->OSTCBID < MIN_ID) {
1855                 MAX_Arrived = ptcb->OSTCBID;
1856                 MIN_ID = ptcb->OSTCBID;
1857                 OSPrioHighRdy = prio;
1858             }
1859             else {
1860                 //printf("Task @%p (prio=%d) is NOT ready\n", ptcb, prio);
1861             }
1862         }
1863         else {
1864             if (InRdyGrp && InRdyTbl) {
1865                 if (DEBUG)
1866                     printf("Task @%p (prio=%d) is READY %d %d %d\n", ptcb, ptcb->OSTCBID, ptcb->TaskPeriodic, ptcb->OSTCBID, ptcb->TaskPeriodic - ptcb->OSTCBID);
1867                 if (ptcb->OSTCBID < MAX_Arrived && ptcb->OSTCBID != 0) {
1868                     MAX_Arrived = ptcb->OSTCBID;
1869                     MIN_ID = ptcb->OSTCBID;
1870                     OSPrioHighRdy = prio;
1871                 }
1872                 else if (ptcb->OSTCBID == MAX_Arrived && ptcb->OSTCBID < MIN_ID && ptcb->OSTCBID != 0) {
1873                     MAX_Arrived = ptcb->OSTCBID;
1874                     MIN_ID = ptcb->OSTCBID;
1875                     OSPrioHighRdy = prio;
1876                 }
1877                 else if (ptcb->OSTCBID == 0 && ptcb->TaskPeriodic <= MAX_Arrived && prio!=0 && ptcb->OSTCBID < MIN_ID) {
1878                     MAX_Arrived = ptcb->TaskPeriodic;
1879                     MIN_ID = ptcb->OSTCBID;
1880                     OSPrioHighRdy = prio;
1881                 }
1882                 else if (prio==0 && MAX_Arrived==0xFFFF) {
1883                     OSPrioHighRdy = prio;
1884                 }
1885             }
1886             else {
1887                 //printf("Task @%p (prio=%d) is NOT ready\n", ptcb, prio);
1888             }
1889         }
1890     }
1891     ptcb = ptcb->OSTCBNext;
1892 }
1893 if (DEBUG)
1894     printf("OSPrIoHighRdy: %d\n", OSPrioHighRdy);
1895 //INT8U y;
1896
1897 //y = OSInMapTbl[OSRdyGrp];
1898 //OSPrIoHighRdy = (INT8U)((y << 3u) + OSInMapTbl[OSRdyTbl[y]]);
1899 #else
1900     INT8U y;
1901     OS_SHTO *ptcb1;
1902     /* We support up to 256 tasks */
1903 }
```

EDF 我主要用上次 FIFO 的專案稍微修改 [OS_SchedNew](#) 的部分，我將選擇最高優先度 TASK 的方式改為檢查哪個 TASK 剩餘的時間最短，並會在 TASK 剛進入新周期和系統剛啟動時做特殊判斷。

PART II

ucos_ii.h

```
67 #define SYSTEM_END_TIME 40
68 #define DEBUG 0
69 #define APERIODIC_FILE_NAME "./Aperiodicjobs.txt"
70 FILE* fp,*fp2;
71 #define INPUT_FILE_NAME "./TaskSet.txt"
72 #define OUTPUT_FILE_NAME "./Output.txt"
```

```
87
88 INT16U TASK_NUMBER, Aperiodic_NUMBER,Sever_U,Sever_Priority,Max_DeadLine;
89 typedef struct listNode //
90 {
91     task_para_set* data;
92     struct listNode* link;
93 } Node, * NodePtr;
94 NodePtr Rdyhead;
95 OS_STK** Task_STK;
96 task_para_set TaskParameter[OS_MAX_TASKS];
97 task para set Aperiodic TaskParameter[OS_MAX_TASKS];
```

新增 Aperiodic job 相關紀錄變數，並用 linklist 紀錄待處理的 Aperiodic job。

OS_SchedNew():

```
1917 if (OSTime == 0) {
1918     if (inRdyGrp && inRdyTbl && ptcb->TASKWorkLoad != 0) {
1919         choosed = 1;
1920         if (DEBUG)
1921             printf("Task @%p (prio=%2d) is READY %d %d %d\n", ptcb, ptcb->OSTCBId, ptcb->TaskPeriodic, ptcb->OSTCBDly, ptcb->TaskPeriodic - ptcb->OSTCBDly);
1922         if (ptcb->OSTCBDly < MAX_Arrived) {
1923             MAX_Arrived = ptcb->OSTCBDly;
1924             MIN_ID = ptcb->OSTCBId;
1925             OSPrioHighRdy = prio;
1926         }
1927         else if (ptcb->OSTCBDly == MAX_Arrived && (ptcb->OSTCBId < MIN_ID || OSPrioHighRdy == Sever_Priority)) {
1928             MAX_Arrived = ptcb->OSTCBDly;
1929             MIN_ID = ptcb->OSTCBId;
1930             OSPrioHighRdy = prio;
1931         }
1932     }
1933     else {
1934         //printf("Task @%p (prio=%2d) is NOT ready\n", ptcb, prio);
1935     }
1936 }
1937 else {
1938     if (inRdyGrp && inRdyTbl && ptcb->TASKWorkLoad != 0) {
1939         if (DEBUG)
1940             printf("Task @%p (prio=%2d) is READY %d %d %d\n", ptcb, ptcb->OSTCBId, ptcb->TaskPeriodic, ptcb->OSTCBDly, ptcb->TaskPeriodic - ptcb->OSTCBDly);
1941         choosed = 1;
1942         if (ptcb->OSTCBDly < MAX_Arrived && ptcb->OSTCBDly != 0) {
1943             MAX_Arrived = ptcb->OSTCBDly;
1944             MIN_ID = ptcb->OSTCBId;
1945             OSPrioHighRdy = prio;
1946         }
1947         else if (ptcb->OSTCBDly == MAX_Arrived && (ptcb->OSTCBId < MIN_ID || OSPrioHighRdy == Sever_Priority) && ptcb->OSTCBDly != 0) {
1948             MAX_Arrived = ptcb->OSTCBDly;
1949             MIN_ID = ptcb->OSTCBId;
1950             OSPrioHighRdy = prio;
1951         }
1952         else if (ptcb->OSTCBDly == 0 && ptcb->TaskPeriodic <= MAX_Arrived && prio != 63 && ptcb->OSTCBId < MIN_ID) {
1953             MAX_Arrived = ptcb->TaskPeriodic;
1954             MIN_ID = ptcb->OSTCBId;
1955             OSPrioHighRdy = prio;
1956         }
1957         else if (prio == 63 && MAX_Arrived == 0xFFFF) {
1958             OSPrioHighRdy = prio;
```

用 EDF 程式的排程策略，但加上判斷 Periodic 要優先 Aperiodic 的條件。

App_hook.c:

```
171 if ((err2 = fopen_s(&fp2, APERIODIC_FILE_NAME, "r")) == 0) {
172     printf("The file 'TaskSet.txt' was opened\n");
173 }
174 else {
175     printf("The file 'TaskSet.txt' was not opened\n");
176 }
177 Aperiodic_NUMBER = 0, j=0;
178 while (!feof(fp2)) {
179     i = 0;
180     memset(str, 0, sizeof(str));
181     fgets(str, sizeof(str) - 1, fp2);
182     ptr = strtok_s(str, " ", &pTmp);
183
184     while (ptr != NULL) {
185         TaskInfo[i] = atoi(ptr);
186         ptr = strtok_s(NULL, " ", &pTmp);
187
188         /* printf("Info: %d\n", task_info[i]); */
189
190         if (i == 0) {
191             Aperiodic_NUMBER++;
192             Aperiodic_TaskParameter[j].TaskID = TaskInfo[i];
193         }
194         else if (i == 1) {
195             Aperiodic_TaskParameter[j].TaskArriveTime = TaskInfo[i];
196         }
197         else if (i == 2) {
198             Aperiodic_TaskParameter[j].TaskExecutionTime = TaskInfo[i];
199         }
200         else if (i == 3) {
201             Aperiodic_TaskParameter[j].TaskPeriodic = TaskInfo[i];
202         }
203         i++;
204     }
205
206     /* Initial Priority */
207     j++;
208 }
209 fclose(fp2);
```

讀取 Aperiodic jobs 檔案並存入變數

OSIntExit():

```
732         if (OSTCBCur->OSTCBPrio>Sever_Priority)
733             printf("'%zd'\tCompletion\ttask(%zd)\t(%zd)\t\task(%zd)\t\tkId\t\tkId\t\tln/A/n", OSTime, OSTCBCur->OSTCBBid, OSTCBCur->JobNum, OSTCBCur->OSTCBBid, OSTCBCur->JobNum +
734                 , OSTime - Rdyhead->data->TaskArriveTime, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBBidly - OSTCBCur->TaskExecutionTime);
735         fprintf(output_fp, "%zd'\tCompletion\ttask(%zd)\t(%zd)\t\task(%zd)\t\tkId\t\tkId\t\tln/A/n", OSTime, OSTCBCur->JobNum, OSTCBCur->OSTCBBid, OSTCBCur->JobNum +
736             , OSTime - Rdyhead->data->TaskArriveTime, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBBidly - OSTCBCur->TaskExecutionTime);
737         Rdyhead = Rdyhead->link;
738     }
739     else {
740         printf("'%zd'\tCompletion\ttask(%zd)\t(%zd)\t\task(%zd)\t\tkId\t\tkId\t\tln/A/n", OSTime, OSTCBCur->OSTCBBid, OSTCBCur->JobNum, OSTCBCur->OSTCBBid, OSTCBCur->JobNum +
741             , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBBidly - OSTCBCur->TaskArriveTime, OSTCBCur->OSTCBBidly);
742         fprintf(output_fp, "%zd'\tCompletion\ttask(%zd)\t(%zd)\t\task(%zd)\t\tkId\t\tkId\t\tln/A/n", OSTime, OSTCBCur->JobNum, OSTCBCur->OSTCBBid, OSTCBCur->JobNum +
743             , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBBidly - OSTCBCur->TaskPeriodic - OSTCBCur->TaskExecutionTime, OSTCBCur->OSTCBBidly);
744     }
745     OSTCBCur->JobNum += 1;
```

判斷 TASK 如果是 Aperiodic job 完成要切換輸出資訊。

```

756 for (INT16U i = 0; i < Aperiodic_NUMBER; i++) {
757     if (Aperiodic_TaskParameter[i].TaskArriveTime == OSTimeGet()) {
758         if (Max_Deadline < OSTime) {
759             Max_Deadline = OSTime;
760             if (Max_Deadline > Aperiodic_TaskParameter[i].TaskExecutionTime * 100 / Sever_U <= Aperiodic_TaskParameter[i].TaskPeriodic) {
761                 Max_Deadline = Max_Deadline > Aperiodic_TaskParameter[i].TaskExecutionTime * 100 / Sever_U;
762                 if (Rdyhead == NULL) {
763                     NodePtr newPtr = (NodePtr)malloc(sizeof(Node));
764                     Rdyhead = newPtr;
765                     Rdyhead->data = &Aperiodic_TaskParameter[i];
766                     Rdyhead->link = NULL;
767                 }
768             }
769             else {
770                 NodePtr t_node = Rdyhead;
771                 while (t_node->link != NULL)
772                 {
773                     t_node = t_node->link;
774                 }
775                 NodePtr newPtr = (NodePtr)malloc(sizeof(Node));
776                 newPtr->data = &Aperiodic_TaskParameter[i];
777                 newPtr->link = NULL;
778                 t_node->link = newPtr;
779             }
780             if (OSTimePri[0][Sever_Priority] <= OSTimePri[0] == 0) {
781                 printf("A2a(taperiodic job(%d) arrives and sets CUS sever's deadline as %d.\n", OSTime, Aperiodic_TaskParameter[i].TaskID, OSTime + Aperiodic_TaskParameter[i].TaskExecutionTime * 100 / Sever_U);
782                 fprintf(output_fp, "A2a(taperiodic job(%d) arrives and sets CUS sever's deadline as %d.\n", OSTime, Aperiodic_TaskParameter[i].TaskID, OSTime + Aperiodic_TaskParameter[i].TaskExecutionTime * 100 / Sever_U);
783             }
784             OSTimePri[0][Sever_Priority] <= OSTimePri[0] <= Aperiodic_TaskParameter[i].TaskExecutionTime * 100 / Sever_U;
785             OSTimePri[0][Sever_Priority] <= TASKMkLoad <= Aperiodic_TaskParameter[i].TaskExecutionTime;
786             OSTimePri[0][Sever_Priority] <= TaskExecutionTime <= Aperiodic_TaskParameter[i].TaskExecutionTime;
787             OSTimePri[0][Sever_Priority] <= TaskPeriodic <= Aperiodic_TaskParameter[i].TaskExecutionTime * 100 / Sever_U;
788         }
789         else {
790             printf("A2a(taperiodic job(%d) arrives. Do nothing.\n", OSTime, Aperiodic_TaskParameter[i].TaskID);
791             fprintf(output_fp, "A2a(taperiodic job(%d) arrives. Do nothing.\n", OSTime, Aperiodic_TaskParameter[i].TaskID);
792         }
793     }
794     else {
795         printf("A2a(taperiodic job(%d) arrives. But can't Scheduled. (miss absolute deadline)\n", OSTime, Aperiodic_TaskParameter[i].TaskID);
796         fprintf(output_fp, "A2a(taperiodic job(%d) arrives. But can't Scheduled. (miss absolute deadline)\n", OSTime, Aperiodic_TaskParameter[i].TaskID);
797     }
798 }
799 }
800 }

```

檢查所有 Aperiodic job 是否該秒抵達，再判斷能否加入工作列。

```

881         if (OSTCBPrioTbl[Sever_Priority] > OSTCBdy == 0 && Rdyhead != NULL) {
882             if (Rdyhead->data->TaskArriveTime != OSTime)
883                 print("%2dAperiodic job(%d) sets CPU server's deadline as %d\n", OSTime->data->TaskID, OSTime->data->TaskID, OSTime->data->TaskExecutionTime * 100 / Sever_U);
884             printf("Output of %2dAperiodic job(%d) sets CPU server's deadline as %d\n", OSTime->data->TaskID, OSTime->data->TaskID, OSTime->data->TaskExecutionTime * 100 / Sever_U);
885             OSTCBPrioTbl[Sever_Priority] = OSTCBdy = Rdyhead->data->TaskExecutionTime;
886             OSTCBPrioTbl[Sever_Priority] = TASKMwKLoad + Rdyhead->data->TaskExecutionTime;
887             OSTCBPrioTbl[Sever_Priority] = TASKExecutionTime = Rdyhead->data->TaskExecutionTime;
888             OSTCBPrioTbl[Sever_Priority] = TASKPeriodic = Rdyhead->data->TaskExecutionTime * 100 / Sever_U;
889         }
890     }
891 }

```

OSTimeTick()

```

1075     if (ptcb->OSTCBPrio == OSPrioCur) {
1076         ptcb->TASKWorkLoad-=1;
1077         if (ptcb->TASKWorkLoad==0) {
1078             if (ptcb->OSTCBPrio == Sever_Priority) {
1079                 printf("%2d\tAperiodic job(%d) is finished\n", OSTime, Rdyhead->data->TaskID);
1080                 fprintf(output_fp, "%2d\tAperiodic job(%d) is finished\n", OSTime, Rdyhead->data->TaskID);
1081             }
1082             OS_ENTER_CRITICAL();
1083             INT8U y = OSTCBCur->OSTCBY;          /* Delay current task */
1084             OSRdyTbl[y] &= (OS_PRIO)~OSTCBCur->OSTCBBitX;
1085             OS_TRACE_TASK_SUSPENDE(OSTCBCur);
1086             if (OSRdyTbl[y] == 0u) {
1087                 OSRdyGrp &= (OS_PRIO)~OSTCBCur->OSTCBBitY;
1088             }
1089             //OS_TRACE_TASK_DLY(ticks);
1090             OS_EXIT_CRITICAL();
1091         }
1092     }
1093 }
1094

```

判斷該秒是否有 Aperiodic job 結束。

OSTaskSwHook()

```

575     else if (OSTCBHighRdy->OSTCBPrio == 63) {
576         if (OSTCBCur->OSTCBPrio == Sever_Priority) {
577             printf("%2d\tcompletion\ttask(%2d)\ttask(%2d)\ttd\tt\ttd\tt\ttd\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBPrio
578                 , OSTime - Rdyhead->data->TaskArriveTime, OSTCBCur->TaskPeriodic - OSTCBCur->TaskExecutionTime, OSTCBCur->OSTCBID);
579             fprintf(output_fp, "%2d\tcompletion\ttask(%2d)\ttask(%2d)\ttd\tt\ttd\tt\ttd\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBPrio
580                 , OSTime - Rdyhead->data->TaskArriveTime, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID - OSTCBCur->TaskExecutionTime, OSTCBCur->OSTCBID);
581             Rdyhead = Rdyhead->link;
582         }
583     }
584     else {
585         printf("%2d\tcompletion\ttask(%2d)\ttask(%2d)\ttd\tt\ttd\tt\ttd\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBPrio
586             , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID - OSTCBCur->TaskExecutionTime, OSTCBCur->OSTCBID);
587         fprintf(output_fp, "%2d\tcompletion\ttask(%2d)\ttask(%2d)\ttd\tt\ttd\tt\ttd\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBPrio
588             , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID - OSTCBCur->TaskExecutionTime, OSTCBCur->OSTCBID);
589         OSTCBCur->JobNum += 1;
590     }
591 }
592
593 if (OSTCBCur->OSTCBPrio == Sever_Priority) {
594     printf("%2d\tcompletion\ttask(%2d)\ttask(%2d)\ttd\tt\ttd\tt\ttd\tt\tt\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBID, OSTCBHighRdy->JobNum
595         , OSTime - Rdyhead->data->TaskArriveTime, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID - OSTCBCur->TaskExecutionTime);
596     fprintf(output_fp, "%2d\tcompletion\ttask(%2d)\ttask(%2d)\ttd\tt\ttd\tt\ttd\tt\tt\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBID, OSTCBHighRdy->JobNum
597         , OSTime - Rdyhead->data->TaskArriveTime, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID - OSTCBCur->TaskExecutionTime);
598     Rdyhead = Rdyhead->link;
599 }
600 else {
601     printf("%2d\tcompletion\ttask(%2d)\ttask(%2d)\ttd\tt\ttd\tt\ttd\tt\tt\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBID, OSTCBHighRdy->JobNum
602         , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID - OSTCBCur->TaskExecutionTime, OSTCBCur->OSTCBID);
603     fprintf(output_fp, "%2d\tcompletion\ttask(%2d)\ttask(%2d)\ttd\tt\ttd\tt\ttd\tt\tt\n", OSTime, OSTCBCur->OSTCBID, OSTCBCur->JobNum, OSTCBHighRdy->OSTCBID, OSTCBHighRdy->JobNum
604         , OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID, OSTCBCur->TaskPeriodic - OSTCBCur->OSTCBID - OSTCBCur->TaskExecutionTime, OSTCBCur->OSTCBID);
605 }
606 OSTCBCur->JobNum += 1;
607 }

```

判斷 TASK 如果是 Aperiodic job 完成要切換輸出資訊。